

ОТЧЕТ

о выполнении практической работы №2.2

Дисциплина «Оптимизация веб-приложений»

Тема: «Контроллер и представление на PHP»

Специальность: 09.02.07 Информационные системы и программирование

Квалификация: Разработчик веб и мультимедийных приложений

Выполнил: Чеголин Алексей Михайлович

Студент группы: 4ИСИП-923

Проверил:

Преподаватель: Рыбаков А.С.

Дата: «29 сентября 2026 г.»

Оценка: _____

Теоретическая часть:

Контроллер — связующее звено, которое соединяет модели, представления и другие компоненты в рабочее приложение. Он отвечает за обработку запросов пользователя: принимает запрос, проверяет полученные данные, передаёт их модели и выбирает представление для вывода.

Контроллер не должен содержать SQL-запросов — их место в моделях. Контроллер не должен содержать HTML и другой разметки — её выносят в представления. В хорошо спроектированном MVC-приложении контроллеры получаются тонкими, в несколько десятков строк: типичная логика выносится в базовые классы. Модели, наоборот, получаются «толстыми»: структура данных и бизнес-логика специфичны для конкретного приложения и занимают основную часть кода.

Представление (View) задаёт внешнее отображение данных, полученных из контроллера и модели. Виды содержат HTML-разметку и небольшие вставки PHP для перебора, форматирования и вывода данных. Они не должны напрямую обращаться к базе данных — этим занимаются модели — и не должны работать с данными, полученными из запроса пользователя: эту задачу выполняет контроллер. Виды обычно делят на общий шаблон, содержащий разметку для всех страниц (шапку и подвал), и части шаблона для вывода конкретных данных или форм.

Ход работы:

Задание 1. Контроллер с выводом разметки напрямую

Создан базовый класс Controller и контроллер ArticleController с двумя действиями: index выводит список статей, show — одну статью. Данные пока захардкожены, базы ещё нет. Контроллер печатает разметку сам, через echo — это и есть состояние, которое предстоит исправить в задании 2.

Все статьи

Первая статья — [читать](#)

Вторая статья — [читать](#)

Третья статья — [читать](#)

Рис. 1 – Список статей: разметка печатается контроллером через echo

Статья #1

Здесь будет содержимое статьи.

[← Назад](#)

Рис. 2 – Страница одной статьи: контроллер выводит HTML вперемешку с данными

Найденные недостатки этого варианта

- разметка находится в контроллере — любая правка оформления требует изменения логики;
- при выводе не выполняется экранирование: если в названии статьи появится HTML, он попадёт на страницу как разметка;
- общий каркас страницы (шапка, подключение стилей) пришлось бы дублировать в каждом действии;
- данные и их внешний вид жёстко связаны: одно и то же действие нельзя показать в двух видах.

Задание 2. Слой представлений и общий шаблон

Введён класс View, который готовит шаблон и подключает его внутрь общего каркаса, и базовый метод view() в контроллере. Контроллер теперь готовит данные и передаёт их в шаблон, а разметку формирует представление. Стили, шапка и заголовок страницы вынесены в layout.php, поэтому описываются один раз.

Как работает вывод: метод View::render() запускает буферизацию вывода, подключает шаблон страницы, забирает полученный HTML в переменную \$content и лишь затем подключает общий шаблон layout.php, который выводит эту переменную внутри себя. Именно поэтому общий каркас один, а шаблонов может быть сколько угодно.

MVC Blog

Все статьи

Первая статья

[Читать →](#)

Вторая статья

[Читать →](#)

Третья статья

[Читать →](#)

Рис. 3 – Список статей после рефакторинга: контроллер передал данные, шаблон отрисовал

MVC Blog

[← Назад](#)

Статья #1

Здесь будет содержимое статьи. Пока это заглушка.

Рис. 4 – Страница статьи после рефакторинга: та же модель вывода, другой шаблон

Контрольные вопросы:

Задание 1

1. Что делает контроллер?

Принимает запрос, определяет, какое действие нужно выполнить, подготавливает данные и передаёт их на отображение. В этом варианте он ещё и печатает разметку, хотя не должен.

2. Как вы думаете, что мы с вами сделали неправильно?

Разметку поместили в контроллер. Из-за этого нарушено разделение ответственности: контроллер занят одновременно логикой и внешним видом, данные не экранируются, а общий каркас страницы пришлось бы повторять в каждом действии.

3. Что не так с этим echo?

Вывод происходит прямо в момент выполнения действия. Из-за этого нельзя ни изменить набор данных перед показом, ни переиспользовать его для другого представления, ни задать заголовки ответа после начала вывода — что и подтвердилось в практике 2.1, где попытка установить код 404 после echo не сработала. Кроме того, в разметку попадают неэкранированные значения.

Задание 2

1. Что именно делает контроллер после рефакторинга?

Формирует набор данных и передаёт его в представление с указанием шаблона. Никакого вывода разметки в контроллере больше нет.

2. Чем теперь занят контроллер, если из него убрали HTML?

Подготовкой данных и выбором представления. В дальнейшем сюда же добавляются проверка входных данных и обращение к модели.

3. Что осталось в контроллере, а что ушло в шаблон?

В контроллере остались массив данных и вызов `view()` с именем шаблона. В шаблон ушли HTML-теги, перебор массива и экранирование значений; в `layout.php` — шапка, заголовок и подключение стилей.

4. Почему layout.php один, а шаблонов может быть много?

Общий каркас страницы одинаков для всего приложения, поэтому описывается один раз в `layout.php`. Различается только содержательная часть, и она вынесена в отдельные шаблоны. Число шаблонов равно числу видов отображения данных.

5. Как layout.php связан с ob_start() и ob_get_clean()?

`ob_start()` включает буферизацию: вывод шаблона не уходит в браузер, а накапливается в буфере. `ob_get_clean()` забирает накопленный HTML в переменную `$content`. После этого подключается `layout.php`, который выводит эту переменную в нужном месте своей разметки.

6. Что было бы, если бы каждый шаблон содержал свой html, head и body?

Каркас страницы пришлось бы дублировать в каждом шаблоне. Любая правка шапки или подключения стилей означала бы правку всех шаблонов, а расхождения между ними приводили бы к разному поведению страниц.

7. Что будет, если завтра дизайнер захочет поменять меню?

Достаточно изменить одно место — `layout.php`. Ни один контроллер и ни один шаблон содержимого при этом не затрагивается: именно в этом и состоит выигрыш от разделения слоёв.

8. А откуда взять статьи, если их станет 10 000?

Из базы данных. Изменится только источник данных: контроллер начнёт получать статьи у модели, а шаблон и общий каркас останутся прежними. Это и разбирается в следующей практике.

Вывод:

В ходе работы контроллер разделён на две части: логику и отображение. Появился слой представлений с общим шаблоном страницы, благодаря чему каркас сайта описывается один раз, а данные не смешиваются с разметкой. Вывод через `echo` заменён на передачу массива данных в шаблон, поэтому контроллер перестал зависеть от того, как именно будут показаны данные.

Практика показала и практическую пользу такого разделения: разметка теперь экранируется через `htmlspecialchars`, а код стал заметно короче. Подготовленная структура каталогов (Core, Controllers, Views) напрямую используется в следующей практике, где к контроллерам добавляется модель и настоящая база данных.